

# Operating Systems

## Week 14 Lecture 14

Junaid Abdul Wahid

# Previous

- Overview of Mass Storage Structure
- Describe the physical structure of secondary storage devices and the effect of a device's structure on its uses
- Explain the performance characteristics of mass-storage devices

# Today Agenda

- Explore the structure of an operating system's I/O subsystem
- Discuss the principles and complexities of I/O hardware
- OS Security

# Overview

I/O management is a major component of operating system design and operation

- Important aspect of computer operation

- I/O devices vary greatly

- Various methods to control them

- Performance management

- New types of devices frequent

Ports, busses, device controllers connect to various devices

**Device drivers** encapsulate device details

- Present uniform device-access interface to I/O subsystem

# I/O Hardware

Incredible variety of I/O devices

- Storage

- Transmission

- Human-interface

Common concepts - signals from I/O devices interface with computer

- Port** – connection point for device

- Bus** - **daisy chain** or shared direct access

  - PCI** bus common in PCs and servers, PCI Express (**PCIe**)

  - expansion bus** connects relatively slow devices

  - Serial-attached SCSI (SAS)** common disk interface

# I/O Hardware (Cont.)

**Controller (host adapter)** – electronics that operate port, bus, device

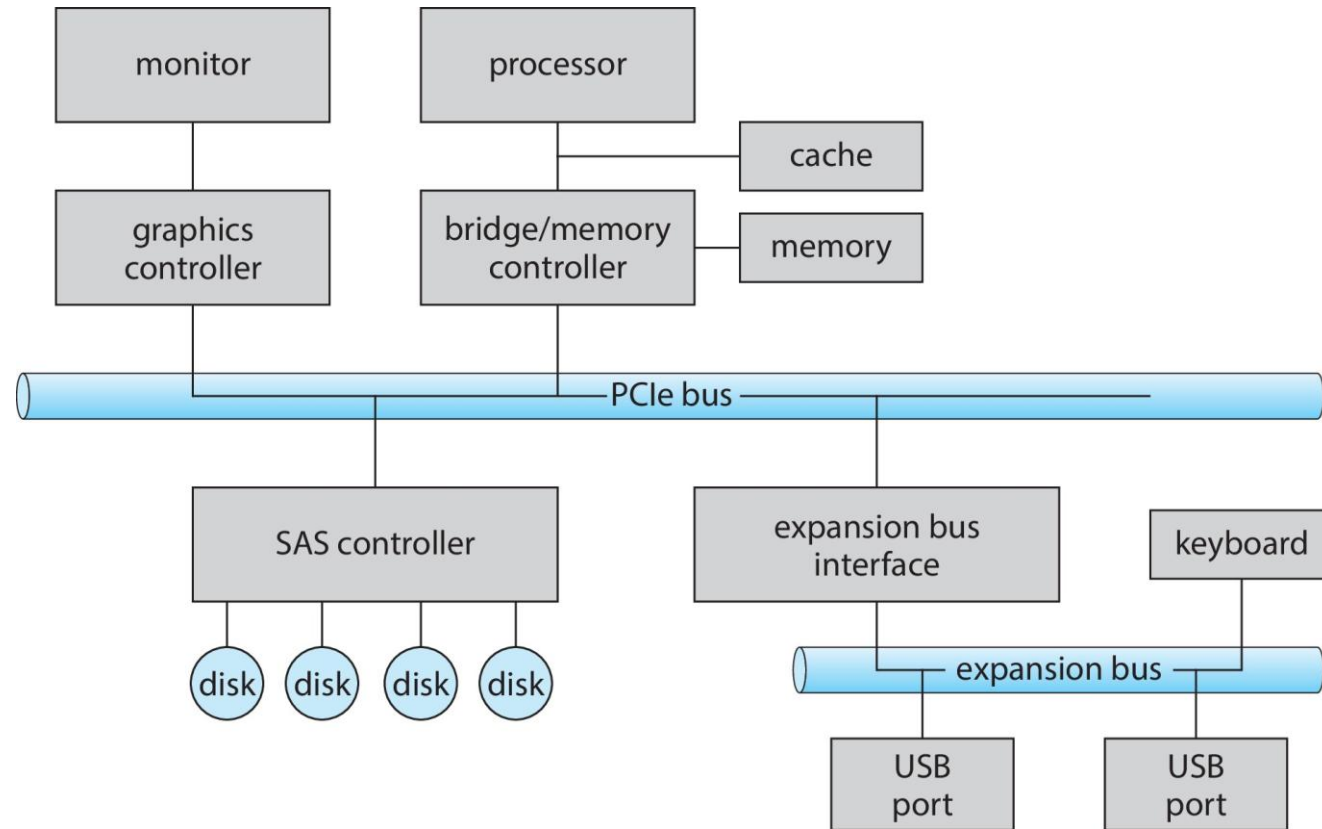
- Sometimes integrated

- Sometimes separate circuit board (host adapter)

- Contains processor, microcode, private memory, bus controller, etc.

  - Some talk to per-device controller with bus controller, microcode, memory, etc.

# A Typical PC Bus Structure



# I/O Hardware (Cont.)

**Fibre channel (FC)** is complex controller, usually separate circuit board (**host-bus adapter, HBA**) plugging into bus

I/O instructions control devices

Devices usually have registers where device driver places commands, addresses, and data to write, or read data from registers after command execution

- Data-in register, data-out register, status register, control register

- Typically 1-4 bytes, or FIFO buffer



# I/O Hardware (Cont.)

Devices have addresses, used by

Direct I/O instructions

## **Memory-mapped I/O**

Device data and command registers mapped to processor address space

Especially for large address spaces (graphics)

# Interrupts

Polling can happen in 3 instruction cycles

- Read status, logical-and to extract status bit, branch if not zero

- How to be more efficient if non-zero infrequently?

CPU **Interrupt-request line** triggered by I/O device

- Checked by processor after each instruction

**Interrupt handler** receives interrupts

- Maskable** to ignore or delay some interrupts

**Interrupt vector** to dispatch interrupt to correct handler

- Context switch at start and end

- Based on priority

- Some **nonmaskable**

- Interrupt chaining if more than one device at same interrupt number

# Direct Memory Access

Used to avoid **programmed I/O** (one byte at a time) for large data movement

Requires **DMA** controller

Bypasses CPU to transfer data directly between I/O device and memory

OS writes DMA command block into memory

- Source and destination addresses

- Read or write mode

- Count of bytes

- Writes location of command block to DMA controller

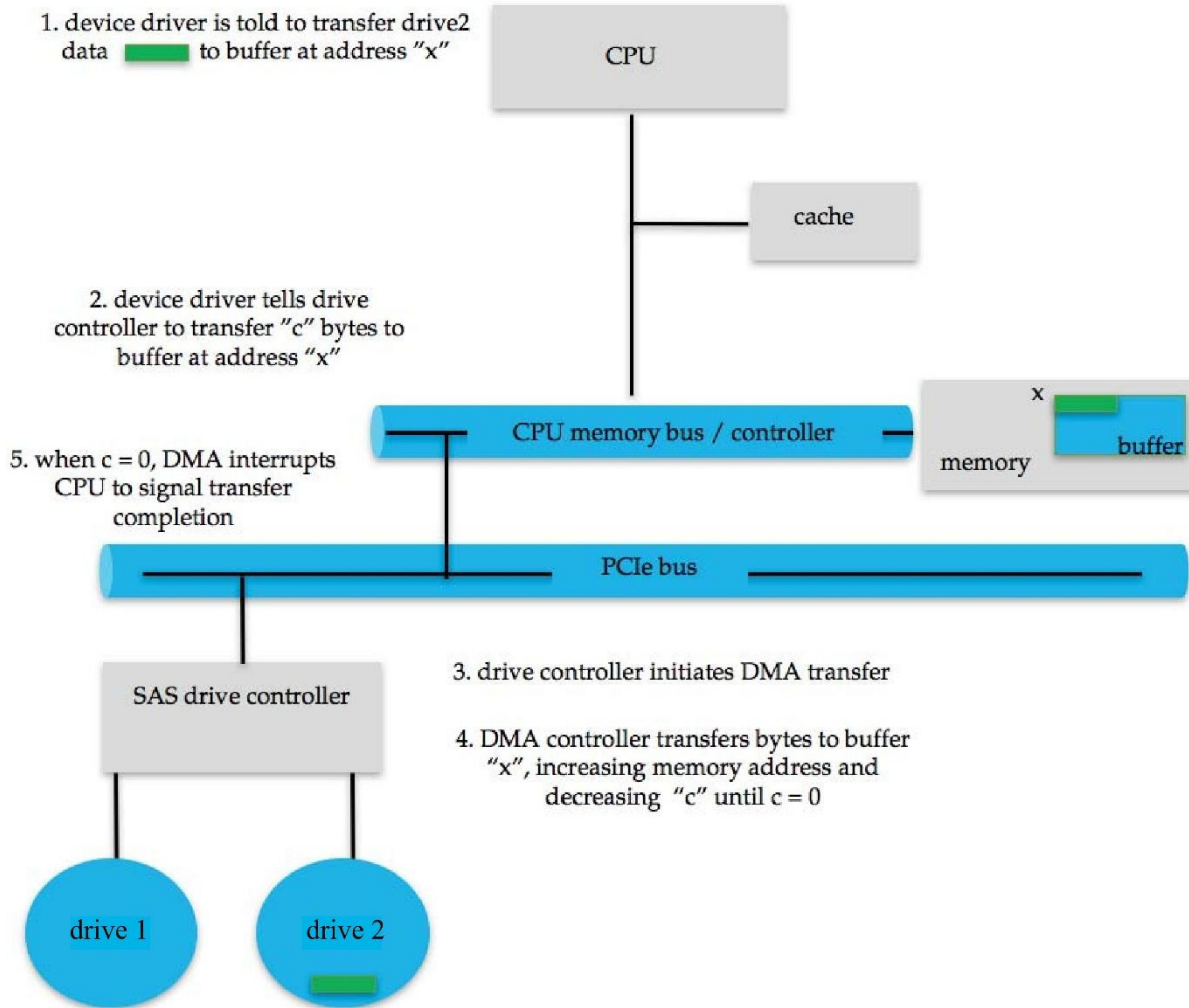
- Bus mastering of DMA controller – grabs bus from CPU

  - Cycle stealing** from CPU but still much more efficient

- When done, interrupts to signal completion

Version that is aware of virtual addresses can be even more efficient - **DVMA**

# Six Step Process to Perform DMA Transfer



# Application I/O Interface

I/O system calls encapsulate device behaviors in generic classes  
Device-driver layer hides differences among I/O controllers from kernel

New devices talking already-implemented protocols need no extra work

Each OS has its own I/O subsystem structures and device driver frameworks

Devices vary in many dimensions

- Character-stream** or **block**

- Sequential** or **random-access**

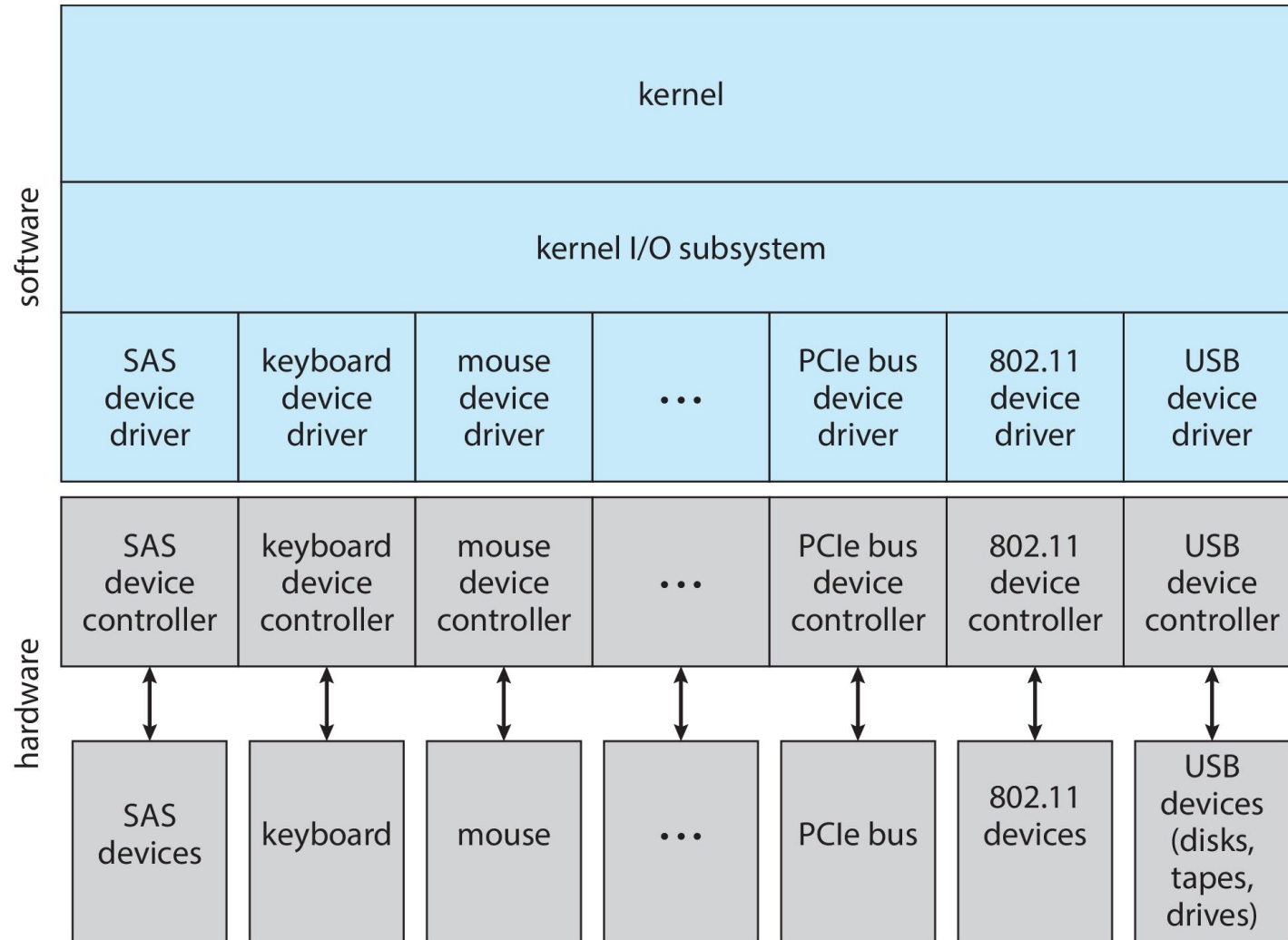
- Synchronous** or **asynchronous** (or both)

- Sharable** or **dedicated**

- Speed of operation**

- read-write, read only, or write only**

# A Kernel I/O Structure



# Characteristics of I/O Devices

| aspect             | variation                                                         | example                               |
|--------------------|-------------------------------------------------------------------|---------------------------------------|
| data-transfer mode | character<br>block                                                | terminal<br>disk                      |
| access method      | sequential<br>random                                              | modem<br>CD-ROM                       |
| transfer schedule  | synchronous<br>asynchronous                                       | tape<br>keyboard                      |
| sharing            | dedicated<br>sharable                                             | tape<br>keyboard                      |
| device speed       | latency<br>seek time<br>transfer rate<br>delay between operations |                                       |
| I/O direction      | read only<br>write only<br>read–write                             | CD-ROM<br>graphics controller<br>disk |

# The Security Problem

System **secure** if resources used and accessed as intended under all circumstances

Unachievable

**Intruders** (**crackers**) attempt to breach security

**Threat** is potential security violation

**Attack** is attempt to breach security

Attack can be accidental or malicious

Easier to protect against accidental than malicious misuse



# Security Violation Categories

## **Breach of confidentiality**

Unauthorized reading of data

## **Breach of integrity**

Unauthorized modification of data

## **Breach of availability**

Unauthorized destruction of data

## **Theft of service**

Unauthorized use of resources

## **Denial of service (DOS)**

Prevention of legitimate use

# Security Measure Levels

Impossible to have absolute security, but make cost to perpetrator sufficiently high to deter most intruders

Security must occur at four levels to be effective:

## **Physical**

Data centers, servers, connected terminals

## **Application**

Benign or malicious apps can cause security problems

## **Operating System**

Protection mechanisms, debugging

## **Network**

Intercepted communications, interruption, DOS

Security is as weak as the weakest link in the chain

Humans a risk too via **phishing** and **social-engineering** attacks

But can too much security be a problem?

# Program Threats

Many variations, many names

## Trojan Horse

Code segment that misuses its environment

Exploits mechanisms for allowing programs written by users to be executed by other users

**Spyware, pop-up browser windows, covert channels**

Up to 80% of spam delivered by spyware-infected systems

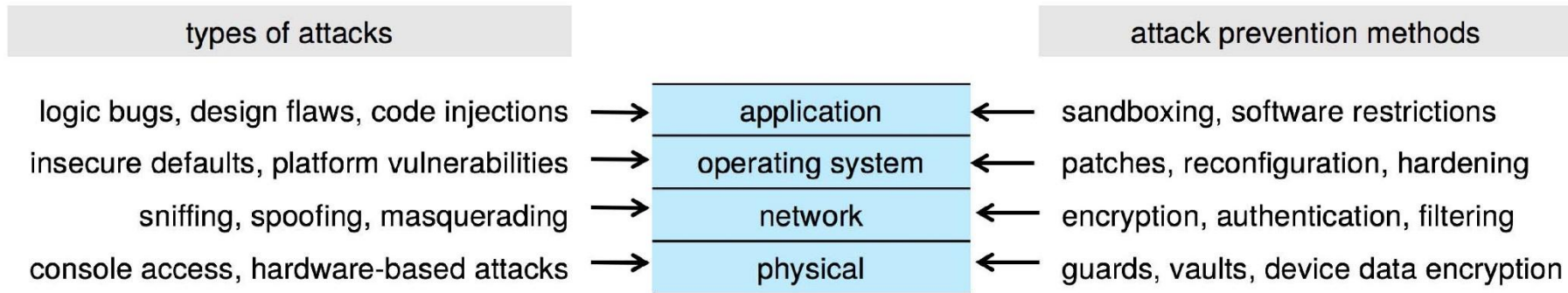
## Trap Door

Specific user identifier or password that circumvents normal security procedures

Could be included in a compiler

How to detect them?

# Four-layered Model of Security



# Program Threats (Cont.)

**Malware** - Software designed to exploit, disable, or damage computer

**Trojan Horse** - Program that acts in a clandestine manner

**Spyware** – Program frequently installed with legitimate software to display ads, capture user data

**Ransomware** – locks up data via encryption, demanding payment to unlock it

Others include trap doors, logic bombs

All try to violate the Principle of Least Privilege

## *THE PRINCIPLE OF LEAST PRIVILEGE*

“The principle of least privilege. Every program and every privileged user of the system should operate using the least amount of privilege necessary to complete the job. The purpose of this principle is to reduce the number of potential interactions among privileged programs to the minimum necessary to operate correctly, so that one may develop confidence that unintentional, unwanted, or improper uses of privilege do not occur.”—Jerome H. Saltzer, describing a design principle of the Multics operating system in 1974: <https://pdfs.semanticscholar.org/1c8d/06510ad449ad24fbdd164f8008cc730cab47.pdf>.

Goal frequently is to leave behind Remote Access Tool (RAT) for repeated access

# System and Network Threats

Some systems “open” rather than **secure by default**

- Reduce **attack surface**

- But harder to use, more knowledge needed to administer

Network threats harder to detect, prevent

- Protection systems weaker

- More difficult to have a shared secret on which to base access

# System and Network Threats (Cont.)

## Port scanning

Automated attempt to connect to a range of ports on one or a range of IP addresses

Detection of answering service protocol

Detection of OS and version running on system

Frequently launched from **zombie systems**

To decrease trace-ability

# System and Network Threats (Cont.)

## Denial of Service

Overload the targeted computer preventing it from doing any useful work

**Distributed Denial-of-Service (DDoS)** come from multiple sites at once

## Port scanning

Automated tool to look for network ports accepting connections

Used for good and evil



# Standard Security Attacks

